claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a541ea58d562f5e01.jsonl`
- SHA-256: `c98301c42dc8dbda22107e8b859c2a8297223cb520c2e1fbc82a400b80f66129`
- Source modified: `2026-06-25T16:56:32+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:54:30.952Z)

PR #380 "feat(O1/P11): Gemini remote-file GC ? startup orphan sweep" is CHECKED OUT (branch o1-p11/gemini-remote-file-gc). It adds a best-effort startup sweep that deletes orphaned Gemini File-API uploads after a hard crash. Changed files (read the real diff with: git diff origin/master...HEAD): backend/providers/gemini.py (new method GeminiProvider.cleanup_orphaned_files(grace_sec=3600) using client.files.list()/delete()); backend/main.py (new _maybe_sweep_gemini_orphans(config), wired in main() next to fail_stale_jobs()); tests/test_provider_gemini.py (+6 tests). CI is GREEN ? do NOT re-run CI/ruff/mypy/tests; review the CODE for correctness, safety, and conventions. Read the actual files + surrounding code to ground every claim.

Review ONLY conventions/scope. Confirm: pure additions (172/0, no formatting churn), single-purpose, the audit tracker docs/CODE_CLEANUP_AUDIT_2026-06-24.md updated for O1/P11, the Verification section present + plausible. Flag scope creep, any hidden behavior change, logging-level issues (failures should be visible per the repo's observability bar), or CODE_MAP placement / convention violations. Report findings + fix.

2. assistant (2026-06-25T16:54:33.911Z)

I'll review PR #380 by reading the actual diff and surrounding code to ground my findings.

3. user (2026-06-25T16:54:35.574Z)

```
 backend/main.py               |  25 ++++++++++
 backend/providers/gemini.py   |  43 ++++++++++++++++++
 tests/test_provider_gemini.py | 104 ++++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 172 insertions(+)
```

4. user (2026-06-25T16:54:36.111Z)

f00b71e feat(O1/P11): Gemini remote-file GC ? startup orphan sweep

5. assistant (2026-06-25T16:54:42.408Z)

Pure additions, single commit, matches the described scope. Now let me read the actual changed files and surrounding code.

6. user (2026-06-25T16:54:43.220Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
```

```
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
 from backend.reporting import emit_indexer_report
 from backend.results import Failure  # standardized error/result contract
 from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
         sys.exit(1)


+def _maybe_sweep_gemini_orphans(config) -> None:
+    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+
 def main():
     parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
     if swept:
         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
     # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
     if args.video_path and not args.server and not args.app:
         print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
 import logging
 import os
 import time
+from datetime import datetime, timedelta, timezone
 from typing import Tuple

 from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
         except Exception:
             pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
```

```
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+                live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue  # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
+            except Exception as e:
+                logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+        if deleted:
+            logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+        return deleted
+
     def analyze_video(self, video_path: str, prompt: str,
                       chunk_duration: float = 0.0,
                       label: str = "") -> Tuple[list, dict]:
diff --git a/tests/test_provider_gemini.py b/tests/test_provider_gemini.py
index 141af81..5df2ef4 100644
--- a/tests/test_provider_gemini.py
+++ b/tests/test_provider_gemini.py
@@ -101,3 +101,107 @@ def test_gemini_provider_upload_fail():
         provider = GeminiProvider(api_key="dummy_api_key", model_name="gemini-2.5-flash")
         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
         assert len(segments) == 0
+
+
+
+# --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
+
+def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
+    f = MagicMock()
+    f.name = name
+    f.mime_type = mime
+    f.create_time = create_time_iso
+    f.state = MagicMock()
+    f.state.name = state
+    return f
```

```
+
+
+def test_cleanup_orphaned_files_deletes_stale_videos_only():
+    """Deletes video files older than the grace window; leaves non-videos and fresh files."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
+    files = [
+        _file_mock("old-vid", "video/mp4", stale),       # stale video -> deleted
+        _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
+        _file_mock("old-img", "image/png", stale),        # non-video -> kept
+        _file_mock("notts", "video/mp4", None),           # no timestamp -> kept (safe)
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1
+    provider._client.files.delete.assert_called_once_with(name="old-vid")
+
+
+def test_cleanup_orphaned_files_list_failure_is_swallowed():
+    """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.side_effect = RuntimeError("network")
+        assert provider.cleanup_orphaned_files() == 0
+
+
+def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
+    """A delete() failure on one file must not abort the sweep."""
+    from datetime import datetime, timedelta, timezone
+    stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
+    files = [
+        _file_mock("a", "video/mp4", stale),
+        _file_mock("b", "video/mp4", stale),
+    ]
+    with patch("backend.providers.gemini.genai"):
+        provider = GeminiProvider(api_key="k", model_name="m")
+        provider._client = MagicMock()
+        provider._client.files.list.return_value = iter(files)
+        # First delete raises, second succeeds ? _safe_delete swallows it.
+        provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
+        deleted = provider.cleanup_orphaned_files(grace_sec=3600)
+    assert deleted == 1  # only the second file counted
+
+
+
+# --- O1/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
+
+def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
+    """A non-Gemini provider must short-circuit (no client built, no sweep)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "openai"
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []  # provider never instantiated
+
```

```
+
+def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
+    """No GEMINI_API_KEY -> no sweep (can't auth)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
+    called = []
+    monkeypatch.setattr(main_mod.provider_registry, "get",
+                        lambda *a, **k: called.append(k) or MagicMock())
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    assert called == []
+
+
+def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
+    """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
startup)."""
+    import backend.main as main_mod
+    cfg = MagicMock()
+    cfg.ai_provider = "gemini"
+    cfg.ai_model = "gemini-2.5-flash"
+    provider = MagicMock()
+    provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
+    monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
+    monkeypatch.setenv("GEMINI_API_KEY", "k")
+    # Must not raise.
+    main_mod._maybe_sweep_gemini_orphans(cfg)
+    provider.cleanup_orphaned_files.assert_called_once()
```

7. assistant (2026-06-25T16:54:45.981Z)

Now let me read the surrounding code in gemini.py to verify conventions (the `_safe_delete`, `client` property, logger setup) and check the test file's use of `_client`.

8. user (2026-06-25T16:54:46.614Z)

```
1       import json
2       import logging
3       import os
4       import time
5       from datetime import datetime, timedelta, timezone
6       from typing import Tuple
7
8       from google import genai
9       from google.genai import types
10
11      from backend.infrastructure.execution_policy import (
12          ExecutionPolicy,
13          PolicyTimeout,
14          poll_until,
15          run_bounded,
16      )
17      from backend.providers.base import AIVideoProvider
18
19      logger = logging.getLogger(__name__)
20
21      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
23      MAX_UPLOAD_MB = 500
24
25      class GeminiProvider(AIVideoProvider):
26          def __init__(self, api_key: str, model_name: str):
27              super().__init__(api_key, model_name)
28              # Initialize client lazily to avoid connection/auth issues during class
```

```
instantiation
29              self._client = None
30
31          @property
32          def client(self):
33              if self._client is None:
34                  self._client = genai.Client(api_key=self.api_key)
35              return self._client
36
37          def _safe_delete(self, video_file) -> None:
38              """Best-effort cleanup of an uploaded File (never raises)."""
39              try:
40                  self.client.files.delete(name=video_file.name)
41              except Exception:
42                  pass
43
44          def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45              """Sweep and delete video files orphaned on the Gemini File API by a previous
46              crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47              leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48              that reach a ``finally``/except).
49
50              Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
51              creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
52              Never raises; returns the count deleted. Intended to run once at startup.
53
54              Args:
55                  grace_sec: only delete files older than this (default 1h). Protects a file a
56                      live worker may still be processing.
57              """
58              cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
59              deleted = 0
60              try:
61                  files = list(self.client.files.list())
62              except Exception as e:
63                  logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
64                  return 0
65              for f in files:
66                  try:
67                      mime = getattr(f, "mime_type", "") or ""
68                      if not mime.startswith("video/"):
69                          continue
70                      # f.create_time is an RFC3339 string on the File API.
71                      ct = getattr(f, "create_time", None)
72                      if ct is None:
73                          # No timestamp ? can't prove it's stale; leave it (safe default).
74                          continue
75                      created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
76                      if created > cutoff:
77                          continue  # too recent ? a live worker may own it
78                      self.client.files.delete(name=f.name)
79                      deleted += 1
80                  except Exception as e:
81                      logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
82              if deleted:
83                  logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
84              return deleted
85
86          def analyze_video(self, video_path: str, prompt: str,
87                            chunk_duration: float = 0.0,
88                            label: str = "") -> Tuple[list, dict]:
```

```
89              log_prefix = f"Gemini {label}" if label else "Gemini"
90
```

9. assistant (2026-06-25T16:54:49.033Z)

Now let me look at the provider_registry and how `provider_registry.get` is used elsewhere, plus the audit doc.

10. user (2026-06-25T16:54:50.608Z)

```
from typing import Dict, Type

from backend.providers.base import AIVideoProvider
from backend.providers.gemini import GeminiProvider


class ProviderRegistry:
    def __init__(self):
        self._registry: Dict[str, Type[AIVideoProvider]] = {}
        # Register Gemini provider
        self.register("gemini", GeminiProvider)

    def register(self, name: str, provider_cls: Type[AIVideoProvider]):
        self._registry[name] = provider_cls

    def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
        if name not in self._registry:
            raise KeyError(f"AI Provider '{name}' is not registered in the ProviderRegistry.
Available: {list(self._registry.keys())}")
        return self._registry[name](api_key=api_key, model_name=model_name)

    def list_available(self) -> Dict[str, str]:
        return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}

provider_registry = ProviderRegistry()
```

11. user (2026-06-25T16:54:52.316Z)

```
2026-06-25 21:30:02 +0530
---DIFF OF AUDIT DOC---
```

12. assistant (2026-06-25T16:54:56.592Z)

Important finding: `provider_registry.get(name, api_key, model_name)` does NOT accept a `model_name` named-arg mismatch ? let me re-check. The call in main.py uses `provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)`. The registry signature is `get(self, name, api_key, model_name)`. That matches. Good.

But critically: the audit doc diff between master and HEAD is EMPTY. The task says the audit tracker `docs/CODE_CLEANUP_AUDIT_2026-06-24.md` should be updated for O1/P11. Let me verify the doc exists and whether it mentions O1/P11.

13. user (2026-06-25T16:54:58.842Z)

```
-rw-r--r-- 1 avidu 197609 28317 Jun 25 22:09 docs/CODE_CLEANUP_AUDIT_2026-06-24.md
=== EXISTS ===
--- search O1/P11 on HEAD ---
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:46:| **D7** | **Absorb relevant BACKLOG items.**
Deferred hardening items from `docs/BACKLOG.md` that overlap with this audit are absorbed into
this project rather than left in BACKLOG. Items that are feature-gaps (not code-cleanup) stay in
BACKLOG. | PR #352 review, 2026-06-24 | D2 and O1 were already independently found; D13 and D14
are new absorptions. The train/eval splits guardrail stays in BACKLOG (it needs a collector-side
manifest change). |
```

HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:88:| **O1** | HIGH | `backend/providers/gemini.py` | **Orphaned remote files on process death.** No GC for uploaded videos on Gemini File API. | 2 |
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:114:| **O1** | "WSL temp/frame/cache + Gemini remote-file GC" | ? ? "staged videos, PNG frame dumps, `*_wasbcache`, and orphaned Gemini uploads are never cleaned" | Phase-2 target. The WSL half is partially handled by `keep_frames=false`; the Gemini half is new. |
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:169:- **Does NOT address any operational risks (O1?O5).** Those need behavioral changes.
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:199:| **P11** | **O1: Gemini remote-file GC** ? cleanup orphaned uploads on process death | P7 | No | ? | ? |
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:239:| 2 | **O1 / P11** | Gemini remote-file GC | The per-call `_safe_delete` on every failure/success path is ALREADY in place (`providers/gemini.py`). REMAINING: a cleanup-on-startup sweep of orphaned files via `client.files.list()` for files left by a crashed previous run. |
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:261:- **Audit source:** 33 findings from a deep whole-repo read on 2026-06-24, plus 2 absorbed from `docs/BACKLOG.md` (D13, D14) for a total of 35. Two additional BACKLOG items (D2, O1) were already independently found and are cross-referenced in §3e.
HEAD:docs/CODE_CLEANUP_AUDIT_2026-06-24.md:273:- `2026-06-24` ? Absorbed 2 findings from `docs/BACKLOG.md` (D13, D14). Cross-referenced D2 and O1 in §3e.

14. assistant (2026-06-25T16:55:02.534Z)

The audit doc exists and references O1/P11, but the PR did NOT update its status tracker ? line 199 still shows `?` (unchecked) for P11, and the audit doc was NOT part of this PR's diff (the diff stat shows only 3 files, none being the doc). Let me look at the relevant section more closely to confirm the tracker convention and whether the PR was supposed to flip it.

15. user (2026-06-25T16:55:03.458Z)

180 | **P0** | **Docs-only PR ? this project doc** + index in `docs/README.md` | ? | Owner approval | ? | [#352](https://github.com/Khelsutra/badminton-highlight-indexer/pull/352) |
181 | **?** | **WASB cache test fix (Windows)** ? 2 pre-existing failures | ? | No | ? | [#353](https://github.com/Khelsutra/badminton-highlight-indexer/pull/353) |
182 | **P4** | **B5: Fix `segmenter_must_be_registered` validator** | P0 | No | ? | [#354](https://github.com/Khelsutra/badminton-highlight-indexer/pull/354) |
183 | **P1** | **D3-a: Convert validators to typed `AppConfig`** (6 files) | P0 | No | ? | [#355](https://github.com/Khelsutra/badminton-highlight-indexer/pull/355) |
184 | **P6a** | **Q3: Deduplicate `_default_db_path`** | P0 | No | ? | [#356](https://github.com/Khelsutra/badminton-highlight-indexer/pull/356) |
185 | **P6b** | **Q4: Cache builtin defaults** | P0 | No | ? | [#358](https://github.com/Khelsutra/badminton-highlight-indexer/pull/358) |
186 | **P6c** | **Q5: requirements.txt torch comment + Q6: isort (I) + Q8: cv2 comment** | P0 | No | ? | [#359](https://github.com/Khelsutra/badminton-highlight-indexer/pull/359) [#360](https://github.com/Khelsutra/badminton-highlight-indexer/pull/360) [#361](https://github.com/Khelsutra/badminton-highlight-indexer/pull/361) |
187 | **T1** | **Tier 1: Q2 (motion return type) + D11 (codec docstring)** | P0 | No | ? | [#362](https://github.com/Khelsutra/badminton-highlight-indexer/pull/362) |
188 | **T2** | **Tier 2: B6 (resolution_fps fallback + warning log)** | P0 | No | ? | [#363](https://github.com/Khelsutra/badminton-highlight-indexer/pull/363) |
189 | **T3** | **Tier 3+4: D12 (sys.path hack) + B3 (PTS gap config) + O5 (lock note) + D7 (centralize TUNED)** | P0 | No | ? | [#364](https://github.com/Khelsutra/badminton-highlight-indexer/pull/364) |
190 | **P6d** | ~~D7: Centralize hardcoded tuning constants~~ ? folded into T3 | P0 | No | ? | [#364](https://github.com/Khelsutra/badminton-highlight-indexer/pull/364) |
191 | **P2** | **D3-b: Convert segmenters to typed `AppConfig`** (15 files: 5 segmenters + 4 detectors + worker + eval + tools) | P1 | No | ? | [#366](https://github.com/Khelsutra/badminton-highlight-indexer/pull/366) |
192 | **P3** | **D3-c: Remove `.get()` compat shim; flip `extra='forbid'`** (18 files, +87/-169) | P2 | No | ? | [#367](https://github.com/Khelsutra/badminton-highlight-indexer/pull/367) |
193 | **P5** | **D2: App-factory refactor** ? replace module-level Optional globals with `create_app(config, db) ? FastAPI` | P3, P4 | No | ? | [#370](https://github.com/Khelsutra/badminton-highlight-indexer/pull/370) |
194 | **P7** | **Phase-1 verification:** full suite green, coverage ? 84%, mypy ratchet |

P1?P6 | No | ? | 1463/1463 pass (#371); `backend.api.job_worker` lifted from mypy quarantine |
195 | ? | ? | ? | ? | ? | ? |
196 | **P8** | **B2: Error-flagging in Gemini provider metrics** (Phase 2) | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
197 | **P9** | **B1: Remove fabricated data from motion segmenter** | P7 | No | ? | ? |
198 | **P10** | **D1: Split `main.py` god-module** (~1450 lines ? smaller modules) | P7 | No
| ? | ? |
199 | **P11** | **O1: Gemini remote-file GC** ? cleanup orphaned uploads on process death |
P7 | No | ? | ? |
200 | **P12** | **D13: ByteTrack deprecation** ? migrate to standalone `trackers` package |
P7 | No | ? | ? |
201 | **P13** | **D4: Deep config merge** ? section-level replace ? recursive merge | P7 |
No | ? | [#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
202 | **P14** | **O2: Single-executor risk** ? documented with mitigations in job_worker.py
| P7 | No | ? | [#374](https://github.com/Khelsutra/badminton-highlight-indexer/pull/374) |
203 | **P15** | **Q7: Type-clean `motion.py`** ? ratchet exercise | P7 | No | ? |
[#373](https://github.com/Khelsutra/badminton-highlight-indexer/pull/373) |
204 | **P17** | **B2 (cont.): finish the `Success`/`Failure` return contract** ? the 4
callers that dropped provider error metrics (`trajectory_hybrid`, `yolo_hybrid`,
`gemini_refine.refine_window`, `gemini_refine.full_video_rallies`) now surface
`metrics["error"]` loudly instead of silently treating a failure as "0 rallies". (Full
`Success`/`Failure` return-type migration of the hybrids is follow-up.) | P8 | No | ? |
[#378](https://github.com/Khelsutra/badminton-highlight-indexer/pull/378) |
205 | **P16+** | Phase 3 items (B4/B7/B8/D5/D6/D8-D10/D14/O3/O4) | P15 | No | ? | ? |
206
207   ---
208
209   ## 7. Open questions
210
211   1. **Q: Should D1 (split `main.py`) be Phase 1 or Phase 2?** ? Leaning Phase 2: D2 (app-
factory) is a natural precursor, and D1 is lower risk after the globals are gone. Owner to
confirm.
212   2. ~~Q: Should Phase 1 include enabling `"I"` (isort) in ruff.toml?~~ ? ? Shipped in
#360.
213
214   ---
215
216   ## 8. Definition of done
217
218   - [x] P0: This doc approved and merged to `master`
219   - [x] P1?P3: Typed-config migration complete ? no `.get()` dict access remains in
backend code; `extra='forbid'` on `AppConfig`
220   - [x] P4: `segmenter_must_be_registered` actually validates against the registry
221   - [x] P5: `create_app()` injects shared state via `app.state` (handlers read
`request.app.state`); `backend.api.job_worker` removed from mypy quarantine. (The Optional
module globals `db_instance`/`global_config` are **retained as a transitional back-compat shim**
and `backend.main` stays mypy-quarantined ? full removal is follow-up, see D1/P10.)
222   - [x] P6: Quick wins Q1?Q6, Q8 landed
223   - [x] P7: Full test suite ? 1463/1463 pass (#371); `ruff` + `mypy` clean;
`backend.api.job_worker` lifted from quarantine
224   - [x] All PRs are single-purpose, branched from `master`, with descriptive commit
messages
225   - [ ] `docs/README.md` updated to reflect the project's terminal state
226   - [ ] Final status flipped to `DONE` and this doc moved to
`docs/archives/past_projects/`
227
228   ---
229
230   ## 8.1 ? Next items ? Phase 2 pick-up (prioritized)
231
232   All items below are **Claude-doable**, small-PR-friendly, and unblocked. Pick up in
order:
233
234   ### Immediate (HIGH impact, low risk)
235

```
236     | # | ID | What | Why |
237     |---|----|------|-----|
238     | ~~1~~ | ~~**B2 / P17**~~ | ~~**Finish** the `Success`/`Failure` return contract~~ ?
**? DONE (this PR).** The 3 callers that dropped provider error metrics now surface
`metrics["error"]` loudly. **Follow-up remains:** migrate the hybrid segmenters' `process_video`
return type from bare `List[Dict]` to the ABC's `Result[...]` (`Success`/`Failure`) ? currently
suppressed with `type: ignore[override]` in `trajectory_hybrid.py` (motion.py is already fully
migrated). |
239     | 2 | **O1 / P11** | Gemini remote-file GC | The per-call `_safe_delete` on every
failure/success path is ALREADY in place (`providers/gemini.py`). REMAINING: a cleanup-on-
startup sweep of orphaned files via `client.files.list()` for files left by a crashed previous
run. |
240
241     ### Near-term (MEDIUM impact)
242
243     | # | ID | What | Why |
244     |---|----|------|-----|
245     | 3 | **B1 / P9** | Remove fabricated data from `motion.py` | Random values for
`server`/`receiver`/`shot_count`/`events`/`avg_speed_kmh` are persisted as real data. Replace
with `None` or honest `"unknown"` markers. |
246     | 4 | **D13 / P12** | ByteTrack deprecation migration | `sv.ByteTrack` removed in
supervision 0.30.0; pinned `<0.30.0` as stopgap. Migrate to standalone `trackers` package. |
247     | 5 | **D1 / P10** | Split `main.py` god-module | ~1450 lines. D2 (app-factory) was the
necessary precursor ? now done. Extract: CLI handlers ? `backend/cli/`, upload routes ? already
in `backend/api/uploads.py`, debug helpers ? `scripts/`. |
248
249     ### Later (deferred)
250
251     | # | ID | What |
252     |---|----|------|
253     | 6+ | **P16+** | B4, B7, B8, D5, D6, D8?D10, D14, O3, O4 ? Phase 3 items |
254
255     > **Removed from this list ? already shipped (see the §6 tracker); they were stale
entries left from before Round 3/4 landed:** D4/P13 deep config merge (#373 ?
`loader.py::_deep_merge`), O2/P14 single-executor risk documented (#374 ? `job_worker.py`),
Q7/P15 `motion.py` type-cleaned (#373 ? no longer in the `mypy.ini` quarantine list).
256
257     ---
258
259     ## 9. References
260
261     - **Audit source:** 33 findings from a deep whole-repo read on 2026-06-24, plus 2
absorbed from `docs/BACKLOG.md` (D13, D14) for a total of 35. Two additional BACKLOG items (D2,
O1) were already independently found and are cross-referenced in §3e.
262     - **Prior hardening sweep:** `docs/CODE_AUDIT_AND_TEST_HARDENING.md` (completed
2026-06-14) ? the previous audit-driven hardening round. This project is the next wave.
263     - **BACKLOG.md** `docs/BACKLOG.md` ? deferred items from the 2026-06-10 sweep; §3e maps
the absorbed items.
264     - **CODE_MAP.md** `docs/CODE_MAP.md` ? placement rules, gotchas, and the cross-cutting
footguns list.
265     - **mypy.ini** ? quarantine list and the ratchet instruction.
266     - **ruff.toml** ? lint rules; `"I"` (isort) deferred.
267     - **PROJECT_DOC_TEMPLATE.md** `docs/PROJECT_DOC_TEMPLATE.md` ? the template this doc
follows.
268
269     ---
```

16. assistant (2026-06-25T16:55:11.384Z)

This confirms a key convention finding. The audit doc is the tracked project doc for this work, and the established convention (visible in every prior row: P8/#373, P13/#373, P14/#374, P17/#378) is that when a tracked item ships, the row is flipped to `?` with the PR link. This PR ships P11/O1 but does NOT update the tracker ? line 199 still shows `?` and line 239 in §8.1 still lists O1/P11 as a "REMAINING" pick-up item. The task explicitly told me to confirm "the audit tracker docs/CODE_CLEANUP_AUDIT_2026-06-24.md updated for O1/P11" ? it is NOT.

Now let me verify the `_safe_delete` test-mock concern: the tests set `provider._client = MagicMock()` then assert `provider._client.files.delete`. Since `cleanup_orphaned_files` uses `self.client` (the property), and `_client` is set, the property returns `_client`. That's correct. But the test comment on line says "_safe_delete swallows it" ? yet `cleanup_orphaned_files` does NOT call `_safe_delete`; it calls `self.client.files.delete` directly inside the try/except. That's just a misleading test comment, minor.

Let me check the CODE_MAP for placement conventions and whether `provider_registry` import ordering is fine, and check the PR description's "Verification section."

17. user (2026-06-25T16:55:13.518Z)

feat(O1/P11): Gemini remote-file GC ? startup orphan sweep
==========

Summary

Closes the **O1/P11** item from `docs/CODE_CLEANUP_AUDIT_2026-06-24.md` (§3c, §8.1 #2).

A hard process death (SIGKILL/OOM/power loss) between `files.upload()` and `_safe_delete()` orphans the uploaded clip on Google's Gemini File-API servers **forever**. The per-call `_safe_delete` (in place since #373) covers every normal/except path, but can't help when the process is killed mid-flight ? those files accumulate across crashed runs with no cleanup.

This adds a best-effort **startup sweep** that reclaims orphaned uploads.

Changes ? pure additions (172 +/0 -)

`backend/providers/gemini.py`
- **`GeminiProvider.cleanup_orphaned_files(grace_sec=3600) -> int`** ? lists all remote files via `client.files.list()`, deletes any whose `mime_type` is `video/*` AND whose `create_time` is older than the grace window (so a live worker mid-upload is never raced). Counts only **successful** deletes (calls `files.delete` directly, not `_safe_delete`, so a failed delete is caught and not counted). Swallows per-file and list-level failures ? **never raises, never blocks startup**.

`backend/main.py`
- **`_maybe_sweep_gemini_orphans(config)`** ? the startup hook. A strict no-op unless `ai_provider == "gemini"` AND `GEMINI_API_KEY` is present. Never raises (a failed sweep is logged + swallowed). Extracted as a standalone function for unit-testability.
- Wired next to `fail_stale_jobs()` in `main()` ? the existing crash-recovery sweep. This is its natural home: both clean up state left by a previous crashed process.

Tests ? +6 new (coverage ratchet held)

GC method (`cleanup_orphaned_files`):
1. `test_cleanup_orphaned_files_deletes_stale_videos_only` ? deletes a stale video; leaves fresh videos (grace), non-videos, and timestamp-less files.
2. `test_cleanup_orphaned_files_list_failure_is_swallowed` ? a `files.list()` failure returns 0, never raises.
3. `test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed` ? a `files.delete()` failure on one file doesn't abort the sweep and isn't counted.

Startup wiring (`_maybe_sweep_gemini_orphans`):
4. `test_maybe_sweep_skips_non_gemini_provider` ? non-Gemini provider ? no client built, no sweep.
5. `test_maybe_sweep_skips_when_no_api_key` ? no `GEMINI_API_KEY` ? no sweep.
6. `test_maybe_sweep_runs_and_swallows_errors` ? Gemini + key ? sweep runs; a provider error is swallowed (never blocks startup).

Safety design
- **Grace-gated** (`grace_sec=3600`, 1h default): only deletes files older than the window, so a concurrent live worker mid-upload is never raced.
- **Video-only**: non-video files (images, etc.) are left untouched ? the sweep only reclaims

what *we* upload.
- **No-timestamp = keep**: a file with no `create_time` can't be proven stale, so it's left (safe default).
- **Never blocks startup**: every failure path (list fail, delete fail, auth fail) is caught + logged at WARNING/DEBUG.

Verification (full local gate, per post-#379 discipline)

- ? `mypy backend --config-file mypy.ini` ? Success, 135 files, no issues
- ? `ruff check .` ? All checks passed
- ? Full suite: **1490 passed**, 10 skipped (up from 1484 ? the 6 new tests)
- ? Diff is pure additions (172 +/0 -) ? no formatting churn

Refs

- Audit tracker: `docs/CODE_CLEANUP_AUDIT_2026-06-24.md` O1/P11 (§3c, §8.1 #2).
- Per-call `_safe_delete` landed in #373; this completes the GC story with the startup orphan sweep.

18. assistant (2026-06-25T16:55:19.215Z)

The PR body is thorough and has a Verification section (mypy, ruff, full suite, pure-additions). It explicitly references the audit tracker. But it does NOT actually update the tracker rows ? that's the gap. The PR claims to "Close the O1/P11 item" but leaves the tracker showing `?` and §8.1 still listing it as remaining.

Now let me verify one concurrency-safety concern more carefully: the grace window protects against racing an upload from *this same machine*, but the deeper question is whether concurrent runs on *different machines sharing one API key* are safe. Let me check the `analyze_video` upload flow to understand how long a file lives during normal processing (could a legitimate in-flight file be older than 1h?).

19. user (2026-06-25T16:55:20.136Z)

```
86          def analyze_video(self, video_path: str, prompt: str,
87                            chunk_duration: float = 0.0,
88                            label: str = "") -> Tuple[list, dict]:
89              log_prefix = f"Gemini {label}" if label else "Gemini"
90
91              metrics: dict = {
92                  "upload_time": 0.0,
93                  "process_time": 0.0,
94                  "gen_time": 0.0,
95              }
96
97              # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
98              try:
99                  size_mb = os.path.getsize(video_path) / (1024 * 1024)
100             except OSError:
101                 size_mb = 0.0
102             if size_mb > MAX_UPLOAD_MB:
103                 logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
104                              f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
105                              f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
106                 metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
107                 metrics["oversize_skipped"] = True
108                 return [], metrics
109
110             # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111             logger.info(f"[{log_prefix}] Uploading {video_path}...")
112             t_upload_start = time.time()
113             video_file = None
```

```
            for attempt in range(3):
                try:
                    video_file = self.client.files.upload(file=video_path)
                    metrics["upload_time"] = time.time() - t_upload_start
                    logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
                    break
                except Exception as e:
                    logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
                    if attempt < 2:
                        time.sleep(2 * (attempt + 1))
            if video_file is None:
                logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
                metrics["error"] = "upload failed after 3 attempts"
                return [], metrics

            # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
            # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
            t_process_start = time.time()
            process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0)  #
20 min cap

            def _processed():
                nonlocal video_file
                video_file = self.client.files.get(name=video_file.name)
                return None if video_file.state.name == "PROCESSING" else video_file

            try:
                video_file = poll_until(_processed, process_policy,
                                        label=f"{log_prefix} processing", logger=logger)
            except PolicyTimeout:
                logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
                self._safe_delete(video_file)
                metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
                return [], metrics
            except Exception as e:
                logger.error(f"[{log_prefix}] Status check failed: {e}")
                self._safe_delete(video_file)
                metrics["error"] = f"status check failed: {e}"
                return [], metrics
            if video_file.state.name == "FAILED":
                logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
                self._safe_delete(video_file)
                metrics["error"] = f"video processing failed: {video_file.error.message}"
                return [], metrics

            metrics["process_time"] = time.time() - t_process_start
            logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")

            # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
            # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
            # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
            logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
            t_gen_start = time.time()
```

```
166             gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167             raw_text = None
168
169             def _generate():
170                 return self.client.models.generate_content(
171                     model=self.model_name,
172                     contents=[video_file, prompt],
173                     config=types.GenerateContentConfig(
174                         response_mime_type="application/json", temperature=0.0),
175                 )
176
177             try:
178                 response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179                 raw_text = response.text
180                 metrics["gen_time"] = time.time() - t_gen_start
181                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182             except Exception as e:                  # PolicyTimeout (hang) or the last 503 ?
both end here
183                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184                 metrics["error"] = f"generate failed after retries: {e}"
185                 metrics["generate_failed"] = True
186             self._safe_delete(video_file)
187
188             if not raw_text:
189                 return [], metrics
190
191             # 4. Parse JSON
192             try:
193                 segments = json.loads(raw_text)
194                 if not isinstance(segments, list):
195                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                     metrics["error"] = "response is not a JSON array"
197                     return [], metrics
198                 return segments, metrics
199             except Exception as e:
200                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
201                 metrics["error"] = f"JSON parse failed: {e}"
202                 return [], metrics
203
```

20. assistant (2026-06-25T16:55:27.378Z)

This is informative for assessing the grace-window safety. The processing poll caps at 1200s (20 min) and the generate at 180s × ~6 retries (~18 min max). So a legitimate in-flight file's lifetime is bounded well under the 1h grace window ? the grace-gate is sound for the same-machine case. The PR's safety reasoning holds.

One concurrency consideration worth noting (but low-severity): the sweep runs on *every* startup of *any* process sharing the API key, and deletes any video >1h old regardless of which machine/run owns it. If a separate long-running job legitimately kept a file alive >1h (not possible given the bounded policies above), it could be deleted. Given the bounded upload?delete window (~max ~38 min), 1h grace is safe. I'll note it as a NIT for documentation, not a real bug.

Now let me verify the import placement convention and that `os` is already imported in main.py (used by `os.environ.get`).

21. user (2026-06-25T16:55:28.604Z)

```
4:import logging
6:import os
35:logger = logging.getLogger(__name__)
```

```
67:from backend.providers import provider_registry
--- import block around line 64 ---
    load_config as load_app_config,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
from backend.providers import provider_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure  # standardized error/result contract
from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
```

22. assistant (2026-06-25T16:55:33.053Z)

Imports are clean: `os` and `logging` already present, the new `provider_registry` import is correctly alphabetized in the `backend.providers` slot. No formatting churn.

Let me verify the `fail_stale_jobs`/`fail_stale_jobs` wiring context in main() to confirm the sweep is placed appropriately and runs on both CLI and server paths (the PR says "video-only" in the main.py comment ? let me confirm what that means and whether the sweep is gated to only run when appropriate).

23. user (2026-06-25T16:55:33.876Z)

```
1320        except OSError as exc:
1321            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1322            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1323            # legitimate single instance).
1324            logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
1325                           "WITHOUT the second-instance guard.", db_path, exc)
1326            _instance_lock = None
1327        else:
1328            if _instance_lock is None:
1329                print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1330                      f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
1331                      f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
1332                sys.exit(1)
1333
1334        db_instance = Database(db_path)
1335
1336        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1337        # the executor is in-process. Mark them failed so pollers see a terminal state.
1338        swept = db_instance.fail_stale_jobs()
1339        if swept:
1340            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1341
1342        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1343        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1344        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1345        _maybe_sweep_gemini_orphans(global_config)
1346
1347        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
```

```
1348        if args.video_path and not args.server and not args.app:
1349            print(f"[CLI] Processing video file: {args.video_path}")
```

24. assistant (2026-06-25T16:55:44.384Z)

The placement is logical ? right after `fail_stale_jobs()`, both being crash-recovery sweeps. It runs before the CLI/server branch, so it executes on every `main()` invocation (CLI, --server, --app). That's intended and correct given the no-op gating inside `_maybe_sweep_gemini_orphans`.

Now I have a complete picture. Let me do a final check: does the network call (`client.files.list()`) on every startup add latency/cost to a user who has Gemini configured but is just running a quick local command? This is a synchronous network round-trip on the startup path. Let me confirm there's no timeout bound on the list call.

The `cleanup_orphaned_files` uses `self.client.files.list()` with no timeout ? a network hang there would block startup until the underlying SDK's default timeout. The function is wrapped in try/except so it won't *crash*, but it could *delay* startup if the network is slow/hanging. This is a real (if minor) observability/latency concern given the repo's "never block startup" claim in the docstring ? "never block startup" is about not *raising*, but a hanging list call could still stall startup. Worth flagging as LOW.

Let me confirm the SDK list call has no bounded timeout in this codebase's usage pattern (the repo wraps other Gemini calls in `run_bounded`/`poll_until` with explicit timeouts ? this one isn't).

25. user (2026-06-25T16:55:44.595Z)

```
15:    run_bounded,
61:            files = list(self.client.files.list())
161:        # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-killer
that was
166:        gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
178:            response = run_bounded(_generate, gen_policy, label=f"{log_prefix} generate",
logger=logger)
```

26. assistant (2026-06-25T16:55:55.833Z)

Confirmed: the codebase's own convention (lines 161-178) is that Gemini network calls get a HARD timeout via `run_bounded`/`ExecutionPolicy` ? the comment even calls it "the hang-killer that was missing." The new `cleanup_orphaned_files` makes an unbounded `client.files.list()` call on the startup path, deviating from this established pattern. Since it runs synchronously before the server/CLI starts, a network hang would stall startup (the try/except prevents a crash but not a hang). This is a LOW/MEDIUM convention-consistency + observability concern.

I now have everything needed. Let me compile findings.

Summary of what I verified:
- **Scope/size**: Pure additions, 172/0, single commit, 3 files ? matches the claim. No formatting churn. Imports correctly placed/alphabetized; `os`/`logging` already present. ?
- **Single-purpose**: Yes ? startup orphan sweep only. No scope creep in code. ?
- **Hidden behavior change**: None ? gated no-op unless `ai_provider == "gemini"` and key present. ?
- **Logging levels**: list-failure ? WARNING (visible), per-file delete-failure ? DEBUG, success ? INFO, main hook failure ? WARNING. Reasonable, mostly meets the observability bar (see one note below).
- **Verification section**: Present and plausible in the PR body (mypy, ruff, 1490 passed/10 skipped, pure additions).
- **Audit tracker updated**: ? NOT updated ? this is the main convention violation.

Findings:

27. user (2026-06-25T16:56:19.391Z)

Structured output provided successfully

28. assistant (2026-06-25T16:56:31.963Z)

Review complete. I read the actual diff (`git diff origin/master...HEAD`), the full `backend/providers/gemini.py` (including the existing `_safe_delete`, `client` property, and the `analyze_video` upload flow that bounds its Gemini calls), the `backend/main.py` wiring next to `fail_stale_jobs()`, the `provider_registry` signature, the audit doc tracker, and the PR body.

Verified positives:
- Pure additions (172/0, 3 files, single commit) ? no formatting churn; imports correctly placed/alphabetized (`os`/`logging` already present).
- Single-purpose, no scope creep in code.
- No hidden behavior change ? `_maybe_sweep_gemini_orphans` is a strict no-op unless `ai_provider == "gemini"` AND `GEMINI_API_KEY` is set; placement next to the existing crash-recovery sweep is the natural home; runs on CLI/--server/--app via the gated hook.
- Safety reasoning is sound: the grace window (1h) comfortably exceeds the bounded normal upload?delete lifetime (processing ?20 min + generate ?~18 min), so a live worker is never raced; video-only and no-timestamp-keep are safe defaults.
- Logging levels are mostly at the right bar (list-failure WARNING, success INFO, main-hook failure WARNING).
- The PR body's Verification section is present and plausible (mypy/ruff/1490 passed-10 skipped/pure-additions).

Key findings reported via StructuredOutput:
- MEDIUM ? the tracked audit doc `docs/CODE_CLEANUP_AUDIT_2026-06-24.md` was NOT updated: row P11 (line 199) still shows `?`/no-PR and §8.1 #2 (line 239) still lists O1/P11 as REMAINING, even though this PR ships it. Every prior shipped item flips its row to `?` with a PR link ? this breaks the tracked-project-doc convention and the PR's own "Closes O1/P11" claim. Fix: flip the row + strike §8.1 #2.
- LOW ? `client.files.list()` at gemini.py:61 is unbounded on the startup path, deviating from the module's own `run_bounded`/`ExecutionPolicy` "hang-killer" convention; "never blocks startup" is only true for *raising*, not *hanging*. Fix: bound it (helper already imported) or soften the wording.
- Two NITs (misleading `_safe_delete` test comment; triple blank lines).